I. Design

1) Design Objective

The design phase defines how the AI-powered employee productivity analytics and burnout prediction system will work end to end. The goal is to create a secure, role-based, scalable platform that supports employee monitoring, manager oversight, HR analytics, and ML-based burnout prediction.

2) System Architecture

The system follows a layered architecture: - **Frontend**: Angular-based UI for login, dashboards, employee views, manager views, and HR views - **Backend**: FastAPI REST API for business logic, authentication, and data access - **Database**: Relational database for users, employees, departments, worklogs, and predictions - **ML Layer**: Python-based prediction engine for burnout analysis - **Authentication**: JWT-based role-aware access control

3) Core Roles and Responsibilities

- **Employee**: Logs in, views own data, and records worklogs
- **Manager**: Views reportees, monitors team activity, and reviews productivity
- **HR**: Views all employees, recent hires, and burnout insights
- **Admin/System**: Manages setup, users, roles, and configuration

4) Functional Design

The system is designed around these major functions: - User authentication and token-based session management - Role-based access to pages and APIs - Employee data management - Manager reportee tracking - HR analytics and recent-hire review - Worklog capture and monitoring - Burnout prediction using ML model output

5) Data Design

Key entities in the system include: - Users - Roles - Employees - Departments - WorkLogs - Predictions

Important relationships: - One user can be linked to one employee profile - One department can have many employees - One manager can have many reportees - One employee can have many worklogs and predictions

6) API Design

The backend is designed as REST APIs with clear responsibilities: - Authentication APIs for login and current-user lookup - Employee APIs for profile and

listing access - Manager APIs for reportee data - HR APIs for recent hires and burnout summaries - Worklog APIs for create/list operations - Prediction APIs for burnout scoring

7) UI/UX Design

The frontend follows a login-first design: - Users see only the login page before authentication - After login, the UI changes based on role - Navigation and pages are shown only if the user has access - HR sees organization-level dashboards - Managers see their team information - Employees see only their own data

8) Security Design

Security is built into the design through: - JWT authentication - Role-based authorization - Protected API routes - Server-side validation - Limited access to sensitive employee data - Recommendation for secure token handling in production

9) ML Design

The ML layer is designed to: - Train on employee productivity and burnout-related features - Store reusable model artifacts - Load the trained model at runtime - Produce burnout predictions for analytics and decision support

10) Deployment Design

The deployment design separates concerns: - Angular frontend runs in the browser - FastAPI backend serves APIs - Database stores application records - ML artifacts are loaded by the backend - The system can be deployed locally for development or on a server for production

11) Design Outcome

The design phase results in: - A clear software architecture - A role-based access model - A normalized data model - Defined REST APIs - A responsive frontend structure - An ML-ready backend for burnout prediction